LAPORAN TUGAS BESAR

IF2111 Algoritma dan Struktur Data STI

PURRMART

Dipersiapkan oleh:

Kelompok 8

Kevin Azra	18223029
Daniel A. M. Sipayung	18223031
Nadia Apsarini Baizal	18223065
Aliya Harta Ary Utama	18223081
Zheannetta Apple Haihando	18223105

Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

Daftar Isi

1 Ringkasan **3**
2 Penjelasan Tambahan Spesifikasi Tugas 3
2.1 Fitur Store List Gacor 3
2.2 Fitur Riwayat Maksimal 3
2.3 Fitur Deteksi Kebocoran Senjata Biologis 3
2.4 Fitur Optimasi Rute Ekspedisi 4
2.5 Fitur Pencarian Barang 4
3 Struktur Data (ADT) 4
3.1 ADT Kustom 4
3.2 ADT Stack 5
3.3 ADT Setmap 6
3.4 ADT Linked List 7
4 Program Utama 8
5 Data Test 9
5.1 PROFILE 9
5.2 CART ADD <nama> <n> 9
5.3 CART REMOVE <nama> <n> 10
5.4 CART SHOW 11
5.5 CART PAY 12
5.6 HISTORY <n> 13
5.7 WISHLIST ADD 14
5.8 WISHLIST SWAP <i> <j> 14
5.9 WISHLIST REMOVE <i> 15
5.10 WISHLIST REMOVE 16
5.11 WISHLIST CLEAR 16
5.12 WISHLIST SHOW 17
6 Test Script 17
7 Pembagian Kerja dalam Kelompok 20
8 Lampiran 21
8.1 Deskripsi Tugas Besar 21
1. About the System 21
2. Menu Program 21
3. Command 22
a. PROFILE 22
b. CART ADD <nama> <n> 22

c. CART REMOVE <nama> <n> 22
d. CART SHOW 23
e. CART PAY 23
f. HISTORY <n> 25
g. WISHLIST ADD 26
h. WISHLIST SWAP <i> <j> 26
i. WISHLIST REMOVE <i> 27
j. WISHLIST REMOVE 27
k. WISHLIST CLEAR 28
4. Perubahan Command 29
a. START, LOAD, dan SAVE 29
b. STORE LIST 29
Konfigurasi Sistem **29**
8.2 Notulen Rapat 30
8.3 Log Activity Anggota Kelompok 33

1 Ringkasan

PURRMART pada aplikasi sebelumnya telah mendapatkan fitur yang sudah menarik dan sangat apik. Namun, aplikasi ini akan melakukan pengembangan lebih lanjut dengan fitur dasar yang sudah diimplementasikan pada PURRMART sehingga pada kesempatan kali ini, kami ingin melakukan pengembangan lebih lanjut dengan menambahkan fitur cart dan wishlist.

Aplikasi ini mensimulasikan berbagai aktivitas jual beli, seperti melihat daftar barang, menambah barang ke keranjang, melakukan pembayaran, mengelola wishlist, hingga menyimpan riwayat pembelian. Sistem diwajibkan menggunakan ADT (Abstract Data Type) tertentu, seperti Stack untuk riwayat pembelian, Setmap untuk keranjang belanja, dan Linked List untuk wishlist.

Terdapat pula opsi bonus, termasuk pencarian barang berbasis jarak Levenshtein, optimasi rute ekspedisi, dan deteksi kebocoran senjata biologis menggunakan algoritma Needleman-Wunsch. Kami juga harus menyertakan driver untuk pengujian, memanfaatkan Makefile untuk kompilasi, dan menjaga struktur program yang rapi.

Dari tugas besar ini, dapat disimpulkan bahwa pengembangan PURRMART dapat diselesaikan dengan menggunakan ilmu pelajaran yang didapat dari mata kuliah IF2111 Algoritma dan Struktur Data.

2 Penjelasan Tambahan Spesifikasi Tugas

2.1 Fitur Store List Gacor

Fitur ini bertujuan untuk mengoptimalkan penyimpanan dan pengelolaan barang di toko. Dalam iterasi sebelumnya, barang disimpan menggunakan List Dinamis yang kurang efisien untuk operasi penyisipan dan penghapusan, terutama jika jumlah barang sangat besar. Fitur ini meminta untuk mengganti implementasi ke struktur data yang lebih efisien yang memungkinkan operasi penyisipan dan penghapusan dengan kompleksitas O(1).

2.2 Fitur Riwayat Maksimal

Fitur ini memperluas kemampuan riwayat pembelian user dengan menyimpan detail lengkap dari setiap transaksi. Sebelumnya, sistem hanya menyimpan nama barang dengan total harga terbesar. Dengan fitur ini, setiap transaksi harus mencatat daftar barang yang dibeli, jumlah masing-masing barang, dan total biaya transaksi. Untuk mendukung fitur ini, format file konfigurasi juga diubah agar data riwayat dapat disimpan secara persisten. Setiap elemen riwayat kini mencakup jumlah barang, total biaya transaksi, dan detail barang seperti nama dan kuantitas. Fitur ini memungkinkan sistem memberikan informasi lebih rinci kepada user.

2.3 Fitur Deteksi Kebocoran Senjata Biologis

Fitur ini menggunakan algoritma Needleman-Wunsch, sistem akan mencocokkan dua sekuens DNA (referensi dan query) dengan menghitung skor kesamaan berdasarkan aturan: +1 untuk match, 0 untuk mismatch, dan -1 untuk gap penalty. Jika skor yang dihasilkan lebih besar dari 80%, maka panjang sekuens yang lebih panjang dianggap terjadi kebocoran. Panjang sekuens maksimum adalah 50 karakter, dan kompleksitas algoritma tidak boleh melebihi O(2n)

untuk memastikan efisiensi. Fitur ini memberikan hasil alignment dan status deteksi kebocoran secara jelas, membantu OWCA memastikan keamanan operasional.

2.4 Fitur Optimasi Rute Ekspedisi

Fitur ini untuk mencari rute pengiriman paling efisien antara beberapa lokasi. Sebelumnya, algoritma BFS digunakan, tetapi karena tidak optimal untuk masalah ini, sistem kini harus mengadopsi algoritma yang lebih efisien, seperti Dijkstra atau Floyd-Warshall. Algoritma Dijkstra cocok untuk menemukan rute terpendek dari satu lokasi ke semua lokasi lainnya, sementara Floyd-Warshall dapat digunakan untuk mencari jalur terpendek antara semua pasangan lokasi. Input fitur ini mencakup jumlah lokasi, rute, dan jarak antar lokasi, sedangkan output-nya adalah rute optimal yang mengunjungi semua lokasi.

2.5 Fitur Pencarian Barang

Fitur ini meningkatkan mesin pencari barang di toko dengan menggantikan pencarian exact match yang kaku menjadi sistem pencarian berbasis kuasi-match. Sistem menggunakan jarak Levenshtein (edit distance) untuk mencocokkan nama barang dan memungkinkan toleransi terhadap perbedaan kecil seperti kesalahan ketik. Barang dianggap cocok jika jarak edit-nya lebih kecil dari atau sama dengan threshold tertentu, dengan default 10. Selain itu, fitur ini mendukung flag opsional seperti -distance untuk mengubah threshold, -ignore-casing untuk mengabaikan perbedaan huruf besar/kecil, dan -ignore-space untuk mengabaikan spasi dalam string pencarian. Dengan fitur ini, user dapat lebih mudah menemukan barang meskipun nama barang tidak ditulis secara presisi.

3 Struktur Data (ADT)

3.1 ADT Kustom

Struktur data yang digunakan untuk menyelesaikan persoalan fitur user dalam PURRMART adalah ADT Kustom yang merepresentasikan data user. ADT ini dirancang untuk menyimpan informasi lengkap mengenai user beserta elemen-elemen penting lainnya, seperti keranjang belanja, riwayat pembelian, dan wishlist. ADT Kustom terdiri dari beberapa elemen utama:

- Nama user, bertipe string yang merepresentasikan identitas user.
- Password, bertipe string untuk autentikasi user.
- money, bertipe integer untuk mencatat jumlah uang yang dimiliki user.
- Keranjang belanja, bertipe Map yang menyimpan nama barang sebagai key dan kuantitas sebagai value.
- Riwayat pembelian, bertipe Stack untuk mencatat daftar transaksi pembelian.
- Wishlist (wishlist), bertipe Linked List untuk menyimpan daftar barang yang diinginkan oleh user.

Beberapa fungsi atau prosedur yang digunakan untuk mengelola ADT ini antara lain:

1. CreateUser(User *U, string name, string password, int money): Membuat user baru dengan nama, password, dan saldo awal tertentu.

2. AddToCart(User *U, string itemName, int quantity): Menambahkan barang ke keranjang belanja user.
3. RemoveFromCart(User *U, string itemName, int quantity): Menghapus barang tertentu dengan kuantitas tertentu dari keranjang belanja.
4. AddToWishlist(User *U, string itemName): Menambahkan barang ke wishlist user.
5. RemoveFromWishlist(User *U, int index): Menghapus barang dari wishlist berdasarkan indeks.
6. AddToHistory(User *U, string itemName, int totalPrice): Menambahkan riwayat pembelian berdasarkan transaksi yang berhasil.

Persoalan yang diselesaikan menggunakan ADT ini adalah persoalan yang terkait dengan pengelolaan akun user, seperti autentikasi, pengelolaan keranjang belanja, pencatatan riwayat pembelian, dan wishlist barang. Alasan pemilihan ADT ini adalah karena struktur datanya yang dirancang sesuai dengan kebutuhan sistem, seperti penggunaan Map untuk memastikan barang dalam keranjang tidak duplikat, Stack untuk menyimpan riwayat pembelian dengan mudah, dan Linked List untuk fleksibilitas manipulasi wishlist. ADT ini mendukung modularitas dan efisiensi dalam implementasi sistem PURRMART.

3.2 ADT Stack

Struktur data yang digunakan untuk menyelesaikan persoalan mengenai fitur tambahan adalah struktur data Stack. Stack merupakan sederetan elemen yang diimplementasikan dalam bentuk list linier yang dikenali berdasarkan elemen puncaknya (TOP). Karena pada Stack hanya diperbolehkan operasi pada address TOP-nya, maka terdapat beberapa aturan penyisipan dan penghapusan yang harus dipenuhi, seperti :

1. Penyisipan selalu dilakukan di atas (TOP)
2. Penghapusan selalu dilakukan pada elemen puncak (TOP).

Stack yang elemennya terdiri atas infotype bertipe Word akan diimplementasikan dalam bentuk ADT stack dengan sketsa struktur data sebagai berikut :

- Memiliki tipe data Stack yang terdiri dari memori tempat penyimpanan elemen infotype bertipe Word sebanyak 100 elemen, serta sebuah integer TOP yang menyatakan alamat/indeks dari elemen puncak Stack.
- Stack merupakan sebuah “tumpukan” yang mengikuti aturan LIFO (Last In First Out) sehingga operasi penambahan dan pengurangan hanya dapat dilakukan di salah satu “ujung” list.
- Memiliki beberapa fungsi/prosedur yang dapat digunakan untuk melakukan operasi terhadap elemen di dalam Stack, yaitu :
    1. CreateEmptyStack(Stack *S) yang berfungsi untuk membuat Stack kosong. Stack disebut kosong jika indeks dari elemen puncaknya (TOP) bernilai Undef (-1).
    2. IsEmptyStack(Stack S) yang akan mengembalikan boolean bernilai true/false untuk memeriksa apakah stack pada parameter merupakan stack kosong atau bukan.
    3. IsFullStack(Stack S) yang akan mengembalikan boolean bernilai true/false untuk memeriksa apakah stack pada parameter sudah penuh atau belum. Stack disebut penuh jika indeks elemen puncaknya sudah menyentuh (MaxEl-1).

4. NbElmtStack(Stack S) yang akan mengembalikan integer yang menyatakan banyaknya elemen dalam stack.
5. Push(Stack *S, infotype X) yang berfungsi untuk menambahkan X sebagai elemen puncak dari Stack. Infotype X akan menjadi elemen puncak yang baru dari Stack.
6. Pop(Stack *S, infotype *X) yang berfungsi untuk menghapus elemen puncak pada Stack. Nilai dari TOP akan disimpan pada X, kemudian indeks TOP akan "turun" ke elemen di bawah TOP sebelumnya.

Persoalan yang diselesaikan menggunakan ADT ini adalah persoalan untuk merepresentasikan urutan yang telah diatur oleh user. ADT ini juga digunakan sebagai konsep dasar dari fitur riwayat pembelian. Alasan mengapa ADT ini digunakan adalah karena konsep yang dimiliki oleh struktur data Stack. Karena elemennya tersusun secara LIFO (Last In First Out), maka hal tersebut dapat mempermudah program untuk menyimpan urutan riwayat pembelian yang sudah dibeli oleh user.

3.3 ADT Setmap

Struktur data yang digunakan untuk menyelesaikan persoalan mengenai pengelolaan keranjang belanja adalah struktur data Setmap. Setmap merupakan sekumpulan elemen yang diimplementasikan dalam bentuk key-value pair, di mana setiap key bersifat unik. Struktur data ini memadukan konsep Set (sekumpulan elemen unik) dan Map (pemetaan elemen berupa pasangan key-value). Dalam konteks PURRMART, key merepresentasikan nama barang, sedangkan value merepresentasikan kuantitas barang.

Setmap akan diimplementasikan dalam bentuk ADT dengan sketsa struktur data sebagai berikut:

- Elemen key bertipe string, digunakan untuk menyimpan nama barang.
- Elemen value bertipe integer, digunakan untuk menyimpan kuantitas barang.
- Memori penyimpanan berupa array statis atau dinamis untuk menyimpan pasangan key-value.
- Ukuran maksimum (MaxEl) untuk menentukan kapasitas maksimum Setmap.

Beberapa fungsi atau prosedur yang digunakan untuk melakukan operasi terhadap elemen dalam Setmap adalah:

1. CreateEmptySetmap(Setmap *S): Membuat Setmap kosong. Setmap disebut kosong jika jumlah elemen di dalamnya bernilai nol.
2. IsEmptySetmap(Setmap S): Mengembalikan nilai boolean untuk memeriksa apakah Setmap kosong.
3. IsFullSetmap(Setmap S): Mengembalikan nilai boolean untuk memeriksa apakah Setmap sudah penuh (jumlah elemen mencapai MaxEl).
4. InsertSetmap(Setmap *S, string key, int value): Menambahkan pasangan key-value ke dalam Setmap. Jika key sudah ada, maka value-nya diperbarui.
5. DeleteSetmap(Setmap *S, string key): Menghapus pasangan key-value berdasarkan key.
6. SearchSetmap(Setmap S, string key): Mencari keberadaan sebuah key dalam Setmap dan mengembalikan indeksnya jika ditemukan.

7. GetValue(Setmap S, string key): Mengembalikan value yang terasosiasi dengan key tertentu.

Persoalan yang diselesaikan menggunakan ADT ini adalah persoalan pengelolaan keranjang belanja, di mana setiap barang dalam keranjang harus unik dan memiliki kuantitas tertentu. Alasan pemilihan ADT Setmap adalah karena sifat unik dari key yang memastikan tidak ada duplikasi barang dalam keranjang, serta efisiensi dalam operasi pencarian, penambahan, dan penghapusan elemen. Struktur ini sangat cocok untuk memetakan nama barang ke kuantitas, sehingga mempermudah pengelolaan keranjang belanja dalam aplikasi PURRMART.

3.4 ADT Linked List

Struktur data yang digunakan untuk menyelesaikan persoalan mengenai fitur wishlist adalah struktur data Linked List. Linked List merupakan sederetan elemen yang diimplementasikan dalam bentuk list linier dinamis, di mana setiap elemen (node) memiliki pointer yang menunjuk ke elemen berikutnya. Linked List cocok digunakan untuk menyimpan daftar elemen dengan jumlah yang dinamis karena memungkinkan penambahan dan penghapusan elemen secara fleksibel tanpa perlu memindahkan elemen lainnya.

Linked List merupakan struktur data dengan sifat *sequential access*, sehingga operasi penyisipan dan penghapusan elemen pada Linked List dapat dilakukan dengan mudah tanpa perlu memindahkan elemen-elemen lain.

Memiliki beberapa fungsi atau prosedur yang dapat digunakan untuk melakukan operasi terhadap elemen dalam Linked List, yaitu:

1. CreateEmptyList(LinkedList *L): Membuat Linked List kosong. Linked List disebut kosong jika pointer first bernilai NULL.
2. IsEmptyList(LinkedList L): Mengembalikan nilai boolean untuk memeriksa apakah Linked List kosong.
3. InsertFirst(LinkedList *L, string X): Menambahkan elemen dengan nilai X pada awal Linked List.
4. InsertLast(LinkedList *L, string X): Menambahkan elemen dengan nilai X pada akhir Linked List.
5. DeleteFirst(LinkedList *L, string *X): Menghapus elemen pertama dari Linked List dan menyimpan nilai elemen yang dihapus pada X.
6. DeleteLast(LinkedList *L, string *X): Menghapus elemen terakhir dari Linked List dan menyimpan nilai elemen yang dihapus pada X.
7. FindElement(LinkedList L, string X): Mencari elemen dengan nilai X dalam Linked List dan mengembalikan pointer ke node tersebut jika ditemukan.
8. DeleteElement(LinkedList *L, string X): Menghapus elemen dengan nilai X dari Linked List jika elemen tersebut ditemukan.

Persoalan yang diselesaikan menggunakan ADT ini adalah persoalan pengelolaan wishlist, di mana user dapat menambahkan, menghapus, menukar, atau menampilkan daftar barang yang diinginkan. Alasan mengapa ADT Linked List dipilih adalah karena sifatnya yang fleksibel dalam mengelola elemen-elemen secara dinamis, serta efisiensi dalam menambahkan atau

menghapus elemen tanpa perlu memindahkan elemen-elemen lain. Hal ini membuat Linked List sangat cocok untuk fitur wishlist yang memerlukan operasi manipulasi daftar secara dinamis dalam aplikasi PURRMART.

4 Program Utama

Program utama PURRMART diimplementasikan dalam file main.c, yang mengintegrasikan seluruh file header dari ADT, modul sistem, driver, dan file konfigurasi yang relevan. Saat program dijalankan, PURRMART menampilkan splash screen dengan ucapan selamat datang kepada user. Setelah tampilan ini, program meminta user untuk memasukkan perintah awal yang terbatas pada START, LOAD <filename.txt>, atau HELP. Jika input yang dimasukkan tidak valid, program akan terus meminta masukan hingga perintah yang valid diberikan.

Perbedaan START dan LOAD terletak pada sumber data yang digunakan. Perintah START akan memuat data dari file default default.txt, sementara LOAD <filename.txt> akan menggunakan data dari file konfigurasi yang ditentukan oleh user. Setelah memuat data, program masuk ke Main Menu.

Pada Main Menu, user dapat menggunakan berbagai perintah yang tersedia untuk menjalankan fitur utama dalam PURRMART. Daftar perintah yang valid dapat ditampilkan dengan memasukkan HELP. Selama user tidak memasukkan perintah QUIT, sistem akan terus berjalan dan menerima input dari user. Beberapa perintah utama yang dapat dijalankan meliputi:

1. PROFILE: Menampilkan data user seperti nama dan saldo.
2. CART ADD <nama> <n>: Menambahkan barang ke keranjang belanja.
3. CART REMOVE <nama> <n>: Menghapus barang dari keranjang belanja.
4. CART SHOW: Menampilkan isi keranjang belanja beserta total harga.
5. CART PAY: Melakukan pembayaran barang dalam keranjang, dengan pengecekan saldo user.
6. WISHLIST ADD : Menambahkan barang ke dalam wishlist user.
7. WISHLIST SWAP <i> <j>: Menukar barang dari posisi i ke j dan sebaliknya.
8. WISHLIST REMOVE <i>: Menghapus barang dari wishlist berdasarkan indeksnya.
9. WISHLIST REMOVE: Menghapus barang dari wishlist berdasarkan nama barang yang dimasukkan
10. WISHLIST CLEAR: Menghapus semua barang yang ada di wishlist
11. WISHLIST SHOW: Menunjukkan barang-barang yang sudah dimasukkan ke dalam wishlist
12. HISTORY <n>: Menampilkan riwayat pembelian terakhir sebanyak n transaksi.

Saat user memasukkan perintah yang valid, PURRMART akan memproses perintah tersebut sesuai dengan logika yang telah diimplementasikan. Sebagai contoh, perintah CART PAY akan memeriksa apakah saldo user mencukupi untuk membeli barang dalam keranjang, dan jika transaksi berhasil, barang tersebut akan dicatat dalam riwayat pembelian user.

Selain fitur utama, PURRMART juga menyediakan fitur bonus, seperti pencarian barang menggunakan jarak Levenshtein dengan perintah STORE FIND, optimasi rute ekspedisi dengan OPTIMASIRUTE, dan deteksi kebocoran senjata biologis dengan algoritma

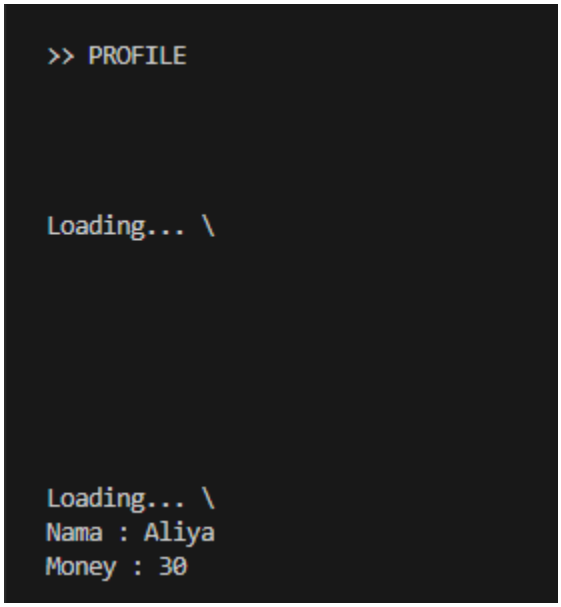
Needleman-Wunsch. Fitur-fitur ini dirancang untuk meningkatkan kompleksitas dan memberikan tantangan tambahan bagi pengembang.

Sistem akan terus berjalan hingga user memasukkan perintah QUIT, yang menghentikan eksekusi program. Program PURRMART memanfaatkan struktur data ADT yang telah diimplementasikan, seperti Stack untuk riwayat pembelian, Setmap untuk keranjang belanja, dan Linked List untuk wishlist, sehingga mendukung pengelolaan data yang efisien dan terorganisasi.

5 Data Test

5.1 PROFILE

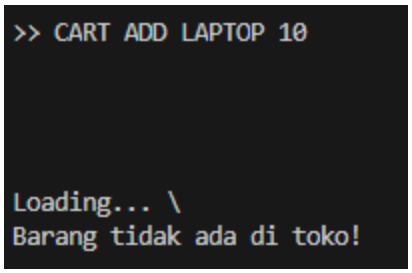
Memastikan bahwa informasi user, seperti nama dan saldo, hanya dapat diakses setelah user berhasil login. Jika user belum login, fitur ini harus menampilkan pesan bahwa akses tidak diperbolehkan



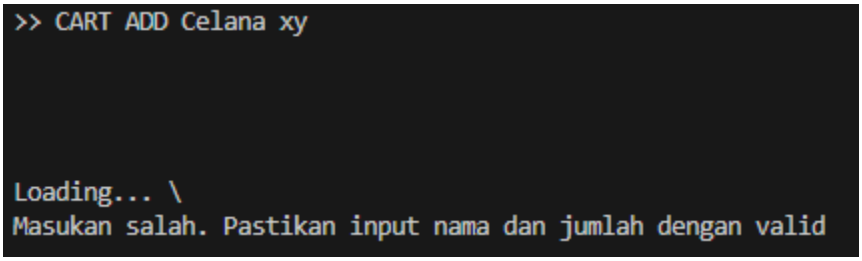
Gambar 5.1.1 Tampilan Profile

5.2 CART ADD <nama> <n>

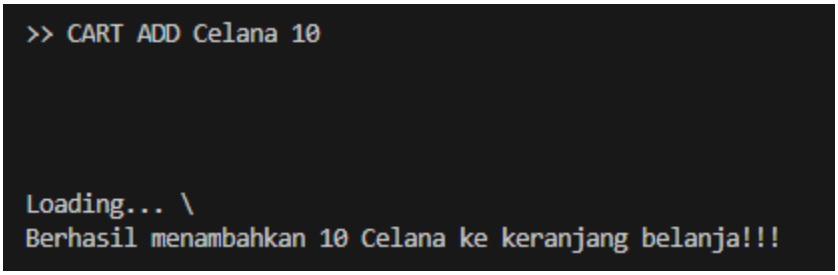
Data test digunakan untuk memvalidasi apakah barang yang dimasukkan ada di toko dan apakah kuantitasnya valid. Jika barang tersedia, maka barang akan berhasil ditambahkan ke keranjang, sedangkan jika tidak ditemukan, akan muncul pesan kesalahan



Gambar 5.2.1 Cart Add Saat Barang Kosong



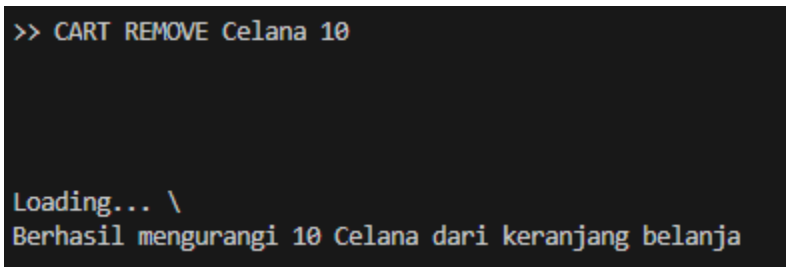
Gambar 5.2.2 Cart Add Saat Inputan Tidak Sesuai



Gambar 5.2.3 Cart Add Saat Berhasil Menambahkan Barang

5.3 CART REMOVE <nama> <n>

Diuji dengan memastikan penghapusan barang dari keranjang hanya dapat dilakukan jika barang ada dan kuantitas yang diminta mencukupi. Jika syarat tersebut tidak terpenuhi, fitur ini akan menampilkan pesan kesalahan.



Gambar 5.3.1 Cart Remove Berhasil Mengurangi Barang

```
>> CART REMOVE Celana ty



Loading... \
Masukan salah. Pastikan input nama dan jumlah dengan valid
```

Gambar 5.3.2 Cart Remove Saat Masukan Tidak Sesuai

```
>> CART REMOVE Celana 100



Loading... \
Tidak berhasil mengurangi, hanya terdapat 10 Celana pada keranjang
```

Gambar 5.3.3 Cart Remove saat Gagal Mengurangi Barang

5.4 CART SHOW

Diuji dengan skenario keranjang kosong dan keranjang berisi, memastikan tampilan daftar barang dan total harga sudah sesuai spesifikasi, serta memberikan pesan yang sesuai jika keranjang kosong.

```
>> CART SHOW



Loading... \
Keranjang belanja kamu kosong
```

Gambar 5.4.1 Tampilan Cart Jika Kosong

```
>> CART SHOW



Loading... \
Berikut adalah isi keranjangmu.
Kuantitas Nama Total
20 Celana 1000
50 Sampo 1000
```

Gambar 5.4.2 Cart Show Saat Barang Tersedia

5.5 CART PAY

Diuji dengan memastikan saldo user cukup atau tidak untuk menyelesaikan pembayaran barang dalam keranjang. Jika saldo mencukupi, transaksi berhasil dilakukan, saldo berkurang, dan barang ditambahkan ke riwayat pembelian. Jika saldo tidak mencukupi, transaksi gagal dengan pesan kesalahan.

```
>> CART PAY



Loading... \
Kamu akan membeli :
Berikut adalah isi keranjangmu.
Kuantitas Nama Total
10 Celana 500
Total biaya yang harus dikeluarkan adalah 500, apakah jadi beli ? (YA/TIDAK)
YA
Selamat kamu telah membeli barang-barang tersebut.
```

Gambar 5.5.1 Cart Pay Saat ingin Membayar

```
>> CART PAY



Loading... \
Keranjang belanja kosong.
```

Gambar 5.5.2 Tampilan Cart Pay Saat Keranjang Kosong

```
>> CART PAY



Loading... \
Kamu akan membeli :
Berikut adalah isi keranjangmu.
Kuantitas Nama Total
10000 Celana 500000
Total biaya yang harus dikeluarkan adalah 500000, apakah jadi beli ? (YA/TIDAK)
YA
Maaf uang kamu tidak cukup.
```

Gambar 5.5.3 Tampilan Cart Pay Saat Saldo Tidak Cukup

5.6 HISTORY <n>

Diuji untuk memastikan bahwa riwayat pembelian ditampilkan dari transaksi terbaru hingga tertua dengan jumlah transaksi sesuai dengan nilai <n>. Jika <n> melebihi jumlah transaksi yang tersedia, seluruh riwayat akan ditampilkan, atau pesan khusus akan muncul jika tidak ada riwayat pembelian.

```
>> CART PAY



Loading... \
Kamu akan membeli :
Berikut adalah isi keranjangmu.
Kuantitas Nama Total
5 Celana 250
Total biaya yang harus dikeluarkan adalah 250, apakah jadi beli ? (YA/TIDAK)
YA
Selamat kamu telah membeli barang-barang tersebut.

>> HISTORY

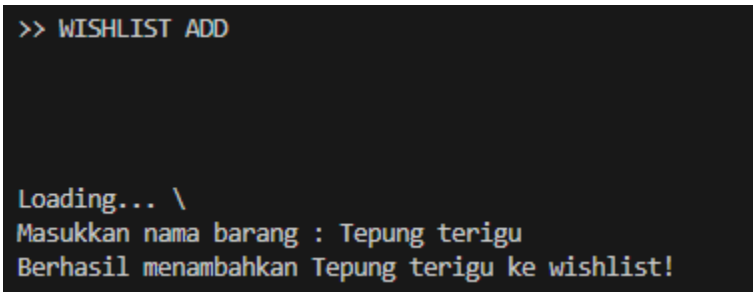


Loading... \
Riwayat Pembelian:
Celana - Total Harga: 50
```

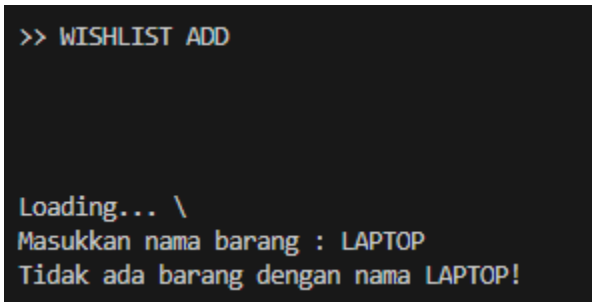
Gambar 5.6.1 Tampilan History

5.7 WISHLIST ADD

Diuji untuk memastikan bahwa barang yang valid dapat ditambahkan ke wishlist, sedangkan barang yang tidak tersedia di toko atau sudah ada di wishlist akan memunculkan pesan kesalahan.



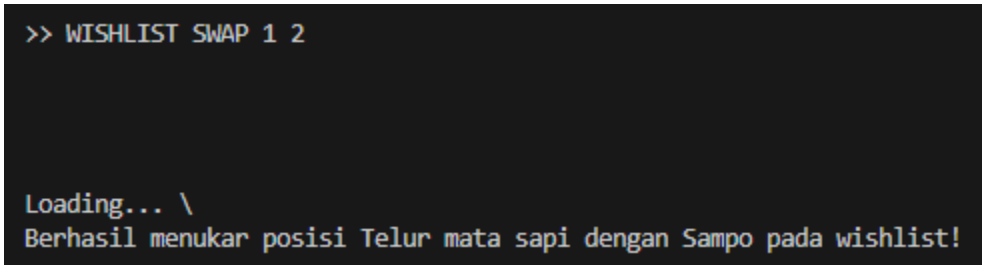
Gambar 5.7.1 Wishlist Add Saat Menambahkan Barang ke Wishlist



Gambar 5.7.2 Wishlist Add Saat Gagal Memasukkan Nama Barang

5.8 WISHLIST SWAP <i> <j>

Diuji untuk memvalidasi pertukaran posisi barang dalam wishlist. Pertukaran hanya akan berhasil jika kedua indeks valid; jika tidak, pesan kesalahan akan ditampilkan.



Gambar 5.8.1 Tampilan Saat Berhasil Menukar Posisi Barang

```
>> WISHLIST SWAP 6 8



Loading... \
Indeks tidak valid untuk melakukan swap atau wishlist kosong!
```

Gambar 5.8.2 Tampilan Saat Indeks Tidak Valid

5.9 WISHLIST REMOVE <i>

Data test memastikan bahwa barang dapat dihapus dari wishlist berdasarkan indeks yang valid, sedangkan indeks yang tidak valid atau wishlist kosong akan menghasilkan pesan kesalahan.

```
>> WISHLIST REMOVE 1



Loading... \
Berhasil menghapus barang posisi ke-1 dari wishlist!

>> WISHLIST SHOW



Loading... \
Berikut adalah isi wishlist-mu:
1 Sampo
```

Gambar 5.9.1 Tampilan Saat Berhasil Menghapus Wishlist pada Posisi Tertentu

```
>> WISHLIST REMOVE 9



Loading... \
Penghapusan barang WISHLIST gagal dilakukan, Barang ke-9 tidak ada di WISHLIST!
```

Gambar 5.9.2 Tampilan Saat Gagal Menghapus Barang di Wishlist pada Posisi Tertentu

5.10 WISHLIST REMOVE

Barang dihapus berdasarkan nama, dan fitur diuji dengan memastikan barang yang valid berhasil dihapus, sementara barang yang tidak ada memunculkan pesan kesalahan.

```
>> WISHLIST REMOVE



Loading... \
Masukkan nama barang yang ingin diremove :
Tepung terigu
Tepung terigu berhasil dihapus dari WISHLIST!
```

Gambar 5.10.1 Tampilan saat Berhasil Menghapus Barang dari Wishlist

```
>> WISHLIST REMOVE



Loading... \
Masukkan nama barang yang ingin diremove :
LAPTOP
Penghapusan barang WISHLIST gagal dilakukan, LAPTOP tidak ada di WISHLIST!
```

Gambar 5.10.2 Tampilan Saat Gagal Menghapus Barang di Wishlist

5.11 WISHLIST CLEAR

Diuji dengan memastikan semua barang di wishlist dihapus, menghasilkan pesan konfirmasi jika wishlist berhasil dikosongkan, atau pesan khusus jika wishlist sudah kosong sebelumnya.

```
>> WISHLIST CLEAR

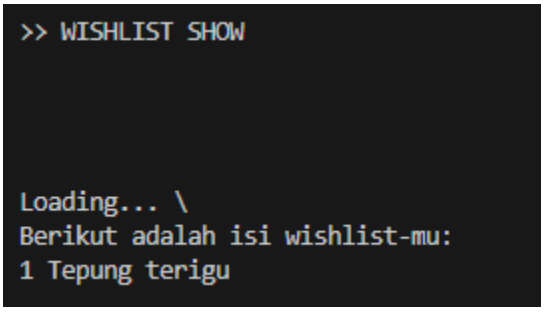


Loading... \
Wishlist telah dikosongkan.
```

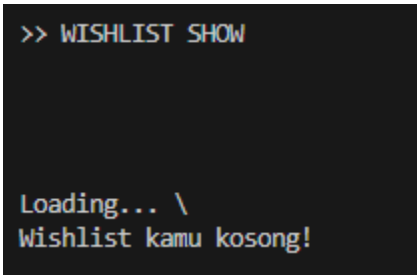
Gambar 5.11.1 Tampilan Saat Seluruh Wishlist Dikosongkan

5.12 WISHLIST SHOW

Diuji untuk memastikan bahwa daftar wishlist ditampilkan dengan benar jika tidak kosong, atau memberikan pesan yang sesuai jika wishlist kosong.



Gambar 5.12.1 Tampilan Saat Menunjukkan Isi Wishlist



Gambar 5.12.2 Tampilan Saat Wishlist yang Ditunjukkan Kosong

6 Test Script

No.	Fitur yang Dites	Tujuan Testing	Langkah-Langkah Testing	Input Data Test	Hasil yang Diharapkan	Hasil yang Keluar
1	Fitur Profile	Memastikan bahwa program dapat menampilkan nama dan uang user	Memasukkan command “PROFILE”	PROFILE	Program mencetak output nama dan uang dari user	Sesuai dengan yang diharapkan
2	Fitur Cart Add <nama> <n>	Memastikan bahwa program dapat menambahkan barang dengan kuantitas tertentu ke dalam	Memasukkan command “CART ADD <nama> <n>” dengan <nama> adalah nama barang yang ingin ditambahkan dan <n> adalah kuantitasnya	CART ADD AK47 20	Program mencetak berhasil menambahkan barang beserta kuantitasnya apabila berhasil. Dan gagal apabila barang yang	Sesuai dengan yang diharapkan

		keranjang belanja			ditambahkan tidak ada di toko	
3	Fitur Cart Remove <nama> <n>	Memastikan bahwa program mengurangi barang sejumlah kuantitas tertentu dari keranjang belanja	Memasukkan command “CART REMOVE <nama> <n>” dengan <nama> adalah nama barang yang ingin dihapus dan <n> adalah kuantitas barang tersebut yang ingin dihapus	CART REMOVE AK47 10	Program mencetak berhasil mengurangi barang beserta kuantitasnya. Namun program mencetak gagal apabila barang ingin dihapus tidak ada di toko	Sesuai dengan yang diharapkan
4	Fitur Cart Show	Memastikan bahwa program menunjukkan barang-barang yang sudah dimasukkan ke dalam keranjang	Memasukkan command “CART SHOW”	CART SHOW	Program mencetak barang-barang yang sudah dimasukkan ke dalam keranjang	Sesuai dengan yang diharapkan
5	Fitur Cart Pay	Memastikan bahwa program membeli barang-barang yang sudah dimasukan ke dalam keranjang	1. Memasukkan command “CART PAY” 2. Konfirmasi dengan “Ya” atau “Tidak” untuk membeli barang	CART PAY Ya	Program mencetak barang yang akan dibeli beserta total biayanya. Dan mengkonfirmasi barang berhasil dibeli atau tidak	Sesuai dengan yang diharapkan
6	Fitur History <n>	Memastikan bahwa program dapat menyimpan dan mencetak riwayat pembelian ke layar sesuai dengan urutan belanja	Memasukkan command “HISTORY <n> ” dengan n berupa integer lebih besar dari 0	HISTORY 3	Program mencetak output riwayat pembelian dengan jumlah yang sesuai dengan n dimana yang baru dibeli terletak di paling atas	Sesuai dengan yang diharapkan
7.	Fitur Wishlist Add	Memastikan bahwa program menambahk	1. Memasukkan command “WISHLIST ADD”	WISHLIST ADD	Program mencetak berhasil menambahkan	Sesuai dengan yang diharapkan

		an suatu barang ke *wishlist*.	2. Memasukkan nama barang	Ayam Geprek Bakar Crispy Besthal	barang ke wishlist	
8.	Fitur Wishlist Swap <i> <j>	Memastikan bahwa program menukar barang posisi ke-i dengan barang posisi ke-j pada *wishlist*	Memasukkan command "WISHLIST SWAP <i> <j> Dengan <i> dan <j> adalah urutan barang pada *wishlist*, urutan dimulai dari 1	WISHLIST SWAP 1 2	Program mencetak berhasil menukar posisi barang pada i menjadi posisi j, dan sebaliknya.	Sesuai dengan yang diharapkan
9.	Fitur Wishlist Remove <i>	Memastikan bahwa program menghapus barang dengan posisi ke-i dari *wishlist*.	Memasukkan command "WISHLIST REMOVE <i>" Dengan <i> adalah posisi barang pada wishlist	WISHLIST REMOVE 2	Program mencetak berhasil menghapus barang pada wishlist di posisi 2	Sesuai dengan yang diharapkan
10.	Fitur Wishlist Remove	Memastikan bahwa program menghapus barang dari wishlist berdasarkan nama barang yang dimasukkan user	1. Memasukkan command "WISHLIST REMOVE" 2. Memasukkan nama barang yang ingin dihapus	WISHLIST REMOVE Labubu	Program mencetak berhasil menghapus Labubu dari wishlist apabila ada di wishlist	Sesuai dengan yang diharapkan
11.	Fitur Wishlist Clear	Memasukkan bahwa program menghapus semua barang yang terdapat di dalam WISHLIST	Memasukkan command "WISHLIST CLEAR"	WISHLIST CLEAR	Program mencetak berhasil mengosongkan wishlist	Sesuai dengan yang diharapkan
12.	Fitur Wishlist Show	Memastikan bahwa program menunjukkan barang-barang yang sudah dimasukkan ke dalam wishlist	Memasukkan command "WISHLIST SHOW"	WISHLIST SHOW	Program mencetak daftar wishlist yang tersedia, apabila daftar kosong maka program mencetak "Wishlist kamu kosong"	Sesuai dengan yang diharapkan

7 Pembagian Kerja dalam Kelompok

Nama Lengkap - NIM	Deskripsi Tugas
Kevin Azra - 18223029	- Membuat code WISHLIST ADD - Membuat code WISHLIST SWAP <i> <j> - Membuat code WISHLIST REMOVE <i> - Membuat code WISHLIST REMOVE - Membuat code WISHLIST CLEAR - Membuat code WISHLIST SHOW - Mengerjakan dokumen bagian ringkasan - Mengerjakan dokumen bagian ADT - Mengerjakan dokumen bagian Data Test - Mengerjakan dokumen bagian Program Utama - Mengerjakan dokumen bagian ringkasan - Mengerjakan dokumen bagian lampiran - Mengatur asistensi dengan asisten
Daniel A. M. Sipayung - 18223031	- Memperbaiki code milestone 1 yang error - Memperbaiki bagian MAIN,WORK, dan WORK CHALLENGE - Merevisi PROFILE - Menyatukan fungsi tambahan - Merapikan header file
Nadia Apsarini Baizal - 18223065	- Memperbaiki code work challenge - Memperbaiki code challenge - Memperbaiki code tebakangka - Memperbaiki code wordl3 - Memperbaiki code quantumwordl3 - Membantu memasukkan code main
Aliya Harta Ary Utama - 18223081	- Membuat fungsi Cart Show - Membuat fungsi Cart Remove - Membuat fungsi Cart Add - Membuat fungsi Cart Pay - Membuat fungsi History - Membuat fungsi Profile - Mendebug All Store code - Merge code to main - Mendebug main code - Mengisi laporan Data Test
Zheannetta Apple Haihando - 18223105	- Membuat perbaikan code Store List - Membuat perbaikan code Store Request

	- Membuat code Store Supply (M1 belum) - Membuat code Store Remove (M1 belum)

8 Lampiran

8.1 Deskripsi Tugas Besar

Spesifikasi Umum

Buatlah sebuah aplikasi simulasi berbasis CLI (*command-line interface*). Sistem ini dibuat dalam **bahasa C** dengan menggunakan **struktur data yang sudah kalian pelajari** di mata kuliah ini. Kalian boleh menggunakan (atau memodifikasi) struktur data yang sudah kalian buat untuk praktikum pada tugas besar ini. Daftar ADT yang wajib digunakan dapat dilihat pada bagian Daftar ADT. *Library* yang boleh digunakan hanya **stdio.h**, **stdlib.h**, **time.h**, dan **math.h**.

System Mechanic

1. About the System

PURRMART adalah sebuah aplikasi yang dapat mensimulasikan aktivitas beli barang pada *e-commerce*. PURRMART memiliki beberapa fitur utama, yaitu:

- Menampilkan barang toko
- Meminta dan menyuplai barang baru ke toko
- Menyimpan dan membeli barang dalam keranjang
- Menampilkan barang yang sudah dibeli
- Membuat dan menghapus *wishlist*
- Bekerja untuk menghasilkan uang

2. Menu Program

Ketika program pertama kali dijalankan, PURRMART akan memperlihatkan *main menu* yang berisi ***welcome menu*** dan beberapa *command* yaitu **START**, **LOAD**, dan juga **HELP**.

Setelah itu, program akan memasuki ***login menu*** yang memiliki command **LOGIN**, **REGISTER**, dan juga **HELP**. Jika pengguna berhasil memasuki kredensial suatu akun, maka mereka akan masuk ke menu selanjutnya.

Main menu menerima masukan berupa *command* yang akan dijelaskan pada bagian berikutnya. Program akan terus menerima *command* sampai diberikan *command* **QUIT** yang berlaku pada seluruh menu.

3. Command

Pengguna dapat memasukkan *command-command* berikut. Seluruh *command* hanya berlaku pada menu utama.

a. PROFILE

PROFILE adalah *command* yang digunakan untuk melihat data diri pengguna. PROFILE hanya dapat dipanggil saat status pengguna telah login

```
>> PROFILE
Nama : Purry
Saldo: 2000

(Silahkan kreasikan atribut yang ditampilkan)
// Kembali ke menu utama
```

b. CART ADD <nama> <n>

CART ADD adalah *command* yang digunakan untuk menambahkan barang dengan kuantitas tertentu ke dalam keranjang belanja.

```
>> CART ADD AK47 20
Berhasil menambahkan 20 AK47 ke keranjang belanja!
// Kembali ke menu utama
```

```
>> CART ADD BebekKaliya 240
Barang tidak ada di toko!
// Perintah invalid; Kembali ke menu utama
```

c. CART REMOVE <nama> <n>

CART REMOVE adalah *command* yang digunakan untuk mengurangi barang sejumlah kuantitas tertentu dari keranjang belanja. Perlu dilakukan validasi terhadap kuantitas yang diberikan, bila kuantitas pada keranjang belanja lebih sedikit dari N maka perintah akan gagal.

```
>> CART REMOVE AK47 10
Berhasil mengurangi 10 AK47 dari keranjang belanja!
// Kembali ke menu utama
```

```
>> CART REMOVE AK47 70
Tidak berhasil mengurangi, hanya terdapat 10 AK47 pada keranjang!
//  Asumsi  di  keranjang  belanja  jumlah  AK47  <70  sehingga  perintah
invalid; Kembali ke menu utama
```

```
>> CART REMOVE BintangSkibidi 70
Barang tidak ada di keranjang belanja!
// Asumsi tidak ada Bintang Skibidi di keranjang belanja; Kembali ke
menu utama
```

d. CART SHOW

CART SHOW adalah *command* yang digunakan untuk menunjukkan barang-barang yang sudah dimasukkan ke dalam keranjang.

```
// Contoh dimana keranjang memiliki isi
>> CART SHOW
Berikut adalah isi keranjangmu.
Kuantitas  Nama    Total
2          AK47    20
1          Lalabu  10
Total biaya yang harus dikeluarkan adalah 30.

// Command mati; Kembali ke menu utama
```

```
// Contoh yang kosong
>> CART SHOW
Keranjang kamu kosong!
```

e. CART PAY

CART PAY adalah *command* yang digunakan untuk membeli barang-barang yang sudah dimasukan ke dalam keranjang. Perlu dipastikan bahwa **pengguna memiliki uang yang cukup** untuk membeli seluruh barang keranjang. Pembelian akan mengurangi uang yang dimiliki pengguna dan menambahkan riwayat pembelian.

Nama barang yang dimasukan ke riwayat pembelian adalah barang dengan total harga (harga barang * kuantitas) terbesar. Jika terdapat lebih dari 1

barang dengan total yang sama, maka yang disimpan adalah barang dengan urutan lexical yang lebih besar. Dimasukan juga total harga pada pembelian tersebut.

```
// Contoh pembayaran yang berhasil (Pengguna memasukan Ya)
>> CART PAY
Kamu akan membeli barang-barang berikut.
Kuantitas  Nama    Total
2          AK47    20
1          Lalabu  10
Total biaya yang harus dikeluarkan adalah 30, apakah jadi dibeli?
(Ya/Tidak): Ya

Selamat kamu telah membeli barang-barang tersebut!

// Command mati; Kembali ke main menu
```

```
// Contoh pembayaran yang gagal (Pengguna tidak memiliki uang yang
cukup)
>> CART PAY
Kamu akan membeli barang-barang berikut.
Kuantitas  Nama    Total
2          AK47    20
1          Lalabu  20
Total biaya yang harus dikeluarkan adalah 40, apakah jadi dibeli?
(Ya/Tidak): Ya

Uang kamu hanya 15, tidak cukup untuk membeli keranjang!

// Command mati; Kembali ke main menu
```

```
// Contoh pembayaran yang gagal (Pengguna memasukan Tidak)
>> CART PAY
Kamu akan membeli barang-barang berikut.
Kuantitas  Nama    Total
2          AK47    20
1          Lalabu  10
Total biaya yang harus dikeluarkan adalah 30, apakah jadi dibeli?
(Ya/Tidak): Tidak

// Command mati; Kembali ke main menu
```

```
// Contoh pembayaran yang gagal (Pengguna memasukan masukan aneh)
>> CART PAY
Kamu akan membeli barang-barang berikut.
Kuantitas  Nama    Total
```

```
2          AK47    20
1          Lalabu  10
Total biaya yang harus dikeluarkan adalah 30, apakah jadi dibeli?
(Ya/Tidak): Purry

// Command mati; Kembali ke main menu
```

```
// Contoh pembayaran yang gagal (Keranjang kosong)
>> CART PAY
Keranjang kamu kosong!
```

f. HISTORY <n>

HISTORY adalah *command* yang digunakan untuk menunjukan riwayat pembelian seorang pengguna. N merupakan jumlah riwayat yang ditampilkan, contoh N=3 maka akan menampilkan 3 riwayat pembelian terbaru. Jika N melebihi jumlah riwayat pembelian yang ada, maka seluruh riwayat pembelian akan ditampilkan. Urutan penunjukan adalah dari yang paling baru ke paling tua.

```
// Contoh menunjukan riwayat pembelian N < total riwayat
>> HISTORY 3
Riwayat pembelian barang:
1. AK47 40
2. AK47 100
3. Lalabu 35

// Command mati; Kembali ke main menu
```

```
// Contoh menunjukan riwayat pembelian N ⩾ total riwayat
>> HISTORY 10
Riwayat pembelian barang:
1. AK47 40
2. AK47 100
3. Lalabu 35
4. AK47 10
5. Meong 500
6. Ayam Goreng Crisbar 20

// Command mati; Kembali ke main menu
```

```
// Contoh riwayat pembelian kosong
Kamu belum membeli barang apapun!
```

g. WISHLIST ADD

WISHLIST ADD merupakan *command* yang digunakan untuk menambahkan suatau barang ke *wishlist*.

```
>> WISHLIST ADD
Masukkan nama barang: Ayam Geprek Bakar Crispy Besthal

Berhasil menambahkan Ayam Geprek Bakar Crispy Besthal ke wishlist!
```

```
>> WISHLIST ADD
Masukkan nama barang: Ayam Geprek Bakar Crispy Besthal

Ayam Geprek Bakar Crispy Besthal sudah ada di wishlist!
```

```
>> WISHLIST ADD
Masukkan nama barang: Ayam Geprek Sambalado Besthal

Tidak ada barang dengan nama Ayam Geprek Sambalado Besthal!
```

h. WISHLIST SWAP <i> <j>

WISHLIST SWAP merupakan *command* yang digunakan untuk menukar barang posisi ke-i dengan barang posisi ke-j pada *wishlist*. Posisi i dan j merupakan urutan barang pada *wishlist*, urutan dimulai dari 1.

```
>> WISHLIST SWAP 1 2
Berhasil menukar posisi Ayam Geprek Bakar Crispy Besthal dengan Ayam Mangut Besthal pada wishlist!
// Urutan Ayam Geprek Bakar Crispy Besthal berubah dari 1 menjadi 2. Sebaliknya, urutan Ayam Mangut Besthal berubah dari 2 menjadi 1
```

```
>> WISHLIST SWAP 1 2
Gagal menukar posisi Ayam Geprek Bakar Crispy Besthal!
// Hanya terdapat satu barang (Ayam Geprek Bakar Crispy Besthal) pada wishlist sehingga posisinya tidak dapat ditukar
```

i. WISHLIST REMOVE <i>

WISHLIST REMOVE adalah *command* yang digunakan untuk menghapus barang dengan posisi ke-i dari *wishlist*.

```
// Contoh menghapus barang ke-i dari WISHLIST
>> WISHLIST REMOVE 2
Berhasil menghapus barang posisi ke-2 dari wishlist!

// Command mati; Kembali ke main menu
```

```
// Contoh penghapusan wishlist yang gagal (Pengguna memasukkan urutan barang yang tidak ada)

//Misalnya dalam kasus ini, hanya terdapat 5 barang di dalam wishlist
>> WISHLIST REMOVE 10
Penghapusan barang WISHLIST gagal dilakukan, Barang ke-10 tidak ada di WISHLIST!

// Command mati; Kembali ke main menu
```

```
// Contoh penghapusan wishlist yang gagal (Wishlist kosong)

//Misalnya dalam kasus ini, tidak ada barang di dalam wishlist
>> WISHLIST REMOVE 1
Penghapusan barang WISHLIST gagal dilakukan, WISHLIST kosong!

// Command mati; Kembali ke main menu
```

```
// Contoh penghapusan wishlist yang gagal (Pengguna memasukkan perintah yang tidak valid)
>> WISHLIST REMOVE XY
Penghapusan barang WISHLIST gagal dilakukan, command tidak valid!

// Command mati; Kembali ke main menu
```

j. WISHLIST REMOVE

WISHLIST REMOVE adalah *command* yang digunakan untuk menghapus barang dari wishlist berdasarkan nama barang yang dimasukkan pengguna.

```
// Contoh menghapus barang “Lalabu” dari WISHLIST
>> WISHLIST REMOVE
Masukkan nama barang yang akan dihapus : Lalabu
Lalabu berhasil dihapus dari WISHLIST!

// Command mati; Kembali ke main menu
```

```
// Contoh penghapusan wishlist yang gagal (Barang tidak ada di WISHLIST)

>> WISHLIST REMOVE
Masukkan nama barang yang akan dihapus : LoremIpsum
Penghapusan barang WISHLIST gagal dilakukan, LoremIpsum tidak ada di WISHLIST!

// Command mati; Kembali ke main menu
```

k. WISHLIST CLEAR

WISHLIST CLEAR adalah *command* yang digunakan untuk menghapus semua barang yang terdapat di dalam WISHLIST.

```
>> WISHLIST CLEAR
Wishlist telah dikosongkan.
```

l. WISHLIST SHOW

WISHLIST SHOW adalah *command* yang digunakan untuk menunjukkan barang-barang yang sudah dimasukkan ke dalam wishlist.

```
>> WISHLIST SHOW
Berikut adalah isi wishlist-mu:
1 Ayam Geprek Bakar Crispy Besthal
2 Ayam Mangut Besthal
3 Karaage Don
4 Torikatsu Don
```

```
>> WISHLIST SHOW
Wishlist kamu kosong!
```

4. Perubahan Command

Terdapat beberapa *command* iterasi sebelumnya yang berubah, yaitu sebagai berikut.

a. START, LOAD, dan SAVE

Terdapat perubahan konfigurasi yang harus ditambahkan dalam implementasi *command* START, LOAD, dan SAVE.

b. STORE LIST

STORE LIST akan menampilkan nama barang yang dijual **beserta harganya**.

```
>> STORE LIST
List barang yang ada di toko :
- Platypus Laser - Harga: 100
- Shrink Ray - Harga: 500
- Net Shooter - Harga: 250
- Camouflage Cloak - Harga: 150
- Sleep Dart Gun - Harga: 300
- Bubble Blaster - Harga: 200
```

```
>> STORE LIST
TOKO KOSONG
```

Konfigurasi Sistem

File konfigurasi akan dibaca saat memulai permainan. File ini menyimpan data-data yang disimpan ketika sistem dijalankan sebelumnya atau data-data *default*. Spesifikasi dari *file* konfigurasi adalah sebagai berikut:

1. Barisan pertama adalah bilangan bulat positif **N** yang menunjukan banyaknya barang di dalam sistem
2. Selanjutnya, sejumlah **N** baris menyatakan nama barang beserta harganya dengan format **<Harga barang> <Nama barang>**
3. Baris selanjutnya adalah bilangan bulat positif **M** yang menunjukan banyaknya pengguna di dalam sistem
4. Selanjutnya, terdapat data M buah pengguna dengan masing-masing spesifikasi berikut:

a. Baris pertama adalah bilangan bulat positif **K** yang menunjukan banyaknya riwayat pengguna tersebut
b. Selanjutnya, sejumlah **K** baris menyatakan nama barang beserta total biaya dengan format **<Total biaya> <Nama barang>**
c. Baris selanjutnya adalah bilangan bulat positif **J** yang menunjukan banyaknya *wishlist* pengguna tersebut
d. Selanjutnya, sejumlah **J** baris menyatakan nama barang dengan format **<Nama barang>**

Berikut adalah contoh file konfigurasi yang dimuat di awal sebuah *session* sebagai inisialisasi:

```
3                                   # Banyaknya barang di dalam toko
10 AK47
20 Lalabu
20 Ayam Goreng Crisbar
500 Meong
2                                   # Banyaknya pengguna di dalam program
100 user1 alstrukdatkeren           # Data pengguna “user1”
6                                   # Banyaknya riwayat pembelian pengguna “user1”
40 AK47
100 AK47
35 Lalabu
10 AK47
500 Meong
20 Ayam Goreng Crisbar
2                                   # Banyaknya wishlist pengguna “admin”
Ayam Goreng Crisbar
AK47
25 user2 kerenbangetkak             # Data pengguna “user1”
0                                   # Banyaknya riwayat pembelian pengguna “user2”
1                                   # Banyaknya wishlist pengguna “user2”
Meong
```

Perlu diperhatikan bahwa antrian permintaan barang DAN keranjang tiap pengguna tidak disimpan di konfigurasi! Jika sistem dimulai, maka antrian dan keranjang akan dibuat lagi dari 0.

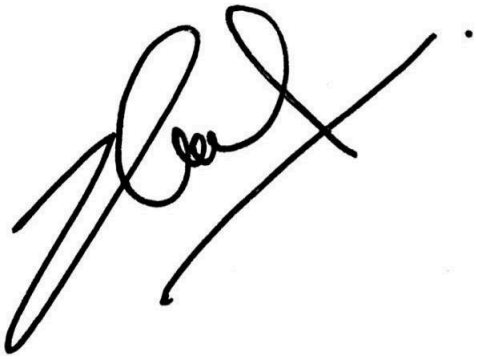
8.2 Notulen Rapat

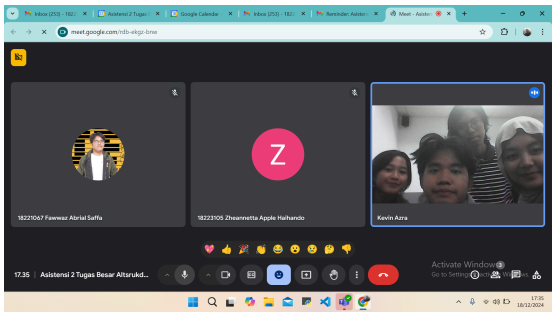
No. Kelompok/Kelas : 08 / 01
Nama Kelompok : ambil ber(k)as
Anggota Kelompok (Nama/NIM) :
1. Kevin Azra / 18223029
2. Daniel A. M. Sipayung / 18223031
3. Nadia Apsarini Baizal / 18223065
4. Aliya Harta Ary Utama / 18223081
5. Zheanetta Apple Haihando / 18223105

Asisten Pembimbing : Fawwaz Abrial Saffa

Asistensi I

Tanggal : 18 Desember 2024 **Tempat : Google Meet** **Kehadiran Anggota Kelompok:** No NIM Tanda tangan 1 18223029  2 18223031 	**Catatan Asistensi:** 1. Progress sudah berkembang dan bagus. 2. Untuk pengumpulan laporan dalam satu folder docs berisi laporan Tugas Besar 1 dan Tugas Besar 2. 3. Untuk demo akan dibuka panjang, jadi tenang saja. 4. Inisiasi Queue, Cart, dan list di main awal. 5. Untuk make file sebaiknya dapat berjalan juga di linux

3 18223065  4 18223081  5 18223105 	

	**Tanda Tangan Asisten:** 

8.3 Log Activity Anggota Kelompok

Tanggal	Aktivitas	Kontributor
9 Desember 2024	Pembagian tugas masing-masing anggota kelompok	Kevin, Aliya, Daniel
12 Desember 2024	Pembuatan template laporan	Kevin
14 Desember 2024	Memulai pengerjaan code	Kevin, Aliya, Daniel, Apple, Nadia
16 Desember 2024	Melanjutkan pengerjaan code dan laporan	Kevin dan Aliya
18 Desember 2024	Melanjutkan pengerjaan code dan laporan	Kevin, Aliya, Daniel, Nadia
18 Desember 2024	Asistensi 1	Kevin, Aliya, Daniel, Nadia
19 Desember 2024	Melanjutkan pengerjaan code dan laporan	Kevin dan Aliya
20 Desember 2024	Melanjutkan pengerjaan code dan membuat Main	Kevin, Aliya, Daniel, Apple, Nadia
21 Desember 2024	Melanjutkan pengerjaan code dan membuat Main	Kevin, Aliya, Nadia, Daniel
22 Desember 2024	Pengumpulan	Kevin, Aliya, Daniel, Apple, Nadia